

FilaFácil: Sistema Distribuído de Fila de Espera para Restaurantes

1. Introdução

Este relatório descreve a arquitetura implementada, as decisões de design adotadas, as dificuldades encontradas ao longo do desenvolvimento e uma reflexão sobre os padrões estudados na disciplina ao longo das quatro sprints do projeto FilaFácil.

O FilaFácil é um sistema distribuído de fila de espera para restaurantes. Seu objetivo é digitalizar o processo de espera por mesas: o cliente entra na fila por um aplicativo móvel, acompanha sua posição em tempo real e é notificado quando sua mesa fica disponível. O operador do restaurante, chamado neste projeto de prestador, gerencia a fila e as mesas por um aplicativo dedicado, recebendo atualizações automáticas sobre novos clientes e chamando o próximo da fila com um toque.

O sistema é composto por quatro componentes principais: um backend REST desenvolvido em Node.js com Express, um middleware orientado a mensagens (MOM) implementado com RabbitMQ, um aplicativo Flutter para o cliente e um aplicativo Flutter para o prestador. Esses componentes se comunicam de forma assíncrona, seguindo o padrão de Arquitetura Orientada a Eventos (EDA).

2. Arquitetura Implementada

A arquitetura do sistema segue o modelo de três camadas clássico estendido com comunicação assíncrona via MOM. O backend centraliza toda a lógica de negócio e serve como única fonte de verdade para o estado do sistema. Os dois aplicativos Flutter funcionam exclusivamente como clientes HTTP, sem acesso direto ao banco de dados ou ao broker de mensagens.

O fluxo principal funciona da seguinte forma: quando um cliente entra na fila, o aplicativo envia uma requisição POST ao backend. O backend valida os dados, persiste a entrada no banco de dados SQLite e, após o commit bem-sucedido, publica um evento na fila `fila.criada.queue` do RabbitMQ. Um worker consumer, rodando como processo separado, consome essa mensagem de forma assíncrona e registra o evento em log. O mesmo padrão se repete quando o prestador chama um cliente (evento `fila.chamada`) e quando o atendimento é finalizado (evento `fila.finalizada`).

Os aplicativos Flutter não se conectam ao RabbitMQ diretamente. A atualização de estado em ambos os apps é feita por polling periódico sobre a API REST. O aplicativo do cliente consulta o status da fila a cada sete segundos e a disponibilidade de mesas a cada cinco

segundos. O aplicativo do prestador consulta fila e mesas em paralelo a cada cinco segundos por meio de um único timer gerenciado pelo `OperadorProvider`.

O banco de dados possui duas tabelas principais. A tabela fila armazena o nome do cliente, o tamanho do grupo, o status atual (AGUARDANDO, CHAMADO ou ATENDIDO) e a referência para a mesa atribuída. A tabela mesa armazena a capacidade e a disponibilidade de cada mesa. A lógica de liberação de mesa é feita automaticamente pelo backend ao marcar uma entrada como ATENDIDO: a mesa correspondente tem seu campo disponível atualizado para verdadeiro na mesma transação.

3. Decisões de Design

Diversas decisões relevantes foram tomadas ao longo do projeto e merecem justificativa explícita.

A primeira decisão foi manter o banco de dados como fonte de verdade e tratar o RabbitMQ como canal de notificação secundário, não como armazenamento de estado. Essa escolha garante que, mesmo que o broker de mensagens esteja indisponível, o sistema continue operando normalmente. A publicação de eventos é feita após o commit no banco e qualquer falha de conexão com o RabbitMQ é descartada silenciosamente. Essa abordagem sacrifica a garantia de entrega de eventos em prol da resiliência do fluxo principal.

A segunda decisão foi optar por polling em vez de WebSocket ou conexão direta com o RabbitMQ nos aplicativos Flutter. A motivação foi principalmente a simplicidade de implementação e a compatibilidade com qualquer ambiente de execução, incluindo emuladores. WebSockets exigiriam gerenciamento de conexão persistente e reconexão automática, o que aumentaria a complexidade sem benefício proporcional para um sistema com baixo volume de atualizações. O intervalo de cinco a sete segundos foi considerado aceitável para o contexto de uso.

A terceira decisão foi criar dois aplicativos Flutter independentes em vez de um único app com autenticação por perfil. Essa separação reflete a diferença de responsabilidades entre os dois atores do sistema: o cliente e o prestador têm fluxos completamente distintos, e um único app com controle de acesso adicionaria complexidade de autenticação que está fora do escopo do projeto. A duplicação de modelos de domínio entre os dois apps foi aceita como custo razoável diante da clareza arquitetural que a separação proporciona.

A quarta decisão foi adotar o banco SQLite como padrão de desenvolvimento, com suporte opcional a PostgreSQL via variável de ambiente. O SQLite elimina a necessidade de um servidor de banco de dados externo para rodar o projeto localmente, reduzindo a barreira de entrada para avaliação. O código de acesso a dados é abstraído por uma camada de repositório que suporta ambos os bancos.

4. Dificuldades Encontradas e Soluções Adotadas

A principal dificuldade técnica durante o desenvolvimento foi a sincronização do estado entre os dois aplicativos Flutter sem comunicação direta entre eles. Como ambos dependem do backend como intermediário, foi necessário garantir que as operações do

prestador (chamar cliente, liberar mesa) refletissem imediatamente no estado retornado pela API, para que o polling do cliente capturasse a atualização no próximo ciclo. Isso foi resolvido mantendo todas as mutações de estado dentro de transações atômicas no banco, garantindo que o GET subsequente sempre retorne o estado pós-mutação.

Uma segunda dificuldade foi rodar os dois aplicativos Flutter simultaneamente em ambientes de emulação para gravar o screencast de demonstração. O Flutter exige que cada instância seja associada a um dispositivo separado. A solução foi utilizar um emulador Android e um segundo dispositivo desktop (Windows) em paralelo, o que permitiu visualizar os dois apps lado a lado durante a gravação.

A terceira dificuldade foi o gerenciamento do ciclo de vida dos timers de polling nos aplicativos Flutter. Em telas que criam novos timers ao serem abertas, foi necessário garantir que o timer anterior fosse cancelado antes de iniciar um novo, evitando acúmulo de timers ativos. Isso foi resolvido adicionando uma chamada a `cancel()` no timer existente antes de cada nova inicialização.

5. Reflexão sobre os Padrões Estudados

5.1. REST (Representational State Transfer)

O estilo arquitetural REST, formalizado por Roy Fielding em sua dissertação de doutorado, define um conjunto de restrições para sistemas hipermídia distribuídos. No FilaFácil, a API REST é o ponto central de integração entre todos os componentes. As rotas seguem a convenção de recursos (`/fila`, `/mesa`), os verbos HTTP são usados de forma semanticamente correta (POST para criação, GET para leitura, PUT para atualização de estado) e o servidor é stateless, ou seja, cada requisição carrega todas as informações necessárias para ser processada sem depender de sessão.

A adoção de REST facilitou o desenvolvimento dos clientes Flutter, pois a interface é previsível e autodocumentada pela semântica dos verbos e dos caminhos. A ausência de estado no servidor também simplifica o escalonamento horizontal, caso o sistema precisasse crescer.

5.2. MOM (Message-Oriented Middleware) e EDA (Event-Driven Architecture)

O padrão MOM desacopla produtores de eventos de seus consumidores, permitindo que cada parte evolua de forma independente. No FilaFácil, o RabbitMQ atua como broker entre o backend e o worker consumer. A cada operação relevante (entrada na fila, chamada de cliente, finalização de atendimento), o backend publica um evento em uma fila dedicada. O consumer processa essas mensagens de forma assíncrona, sem bloquear o fluxo principal da API.

A Arquitetura Orientada a Eventos (EDA) complementa o MOM ao definir que o estado do sistema avança em resposta a eventos discretos e imutáveis. Essa abordagem cria um log natural de todas as transições de estado do sistema, o que facilita auditoria, depuração e eventual reprocessamento de eventos. No contexto deste projeto, os eventos do RabbitMQ

funcionam como um registro assíncrono das operações realizadas, mesmo que o consumer atual apenas registre os eventos em log.

5.3. Clean Architecture

A Clean Architecture, proposta por Robert C. Martin, organiza o código em camadas concêntricas onde as dependências apontam sempre em direção ao centro, ou seja, as camadas externas dependem das internas, nunca o contrário. Tanto o aplicativo do cliente quanto o do prestador foram estruturados com quatro camadas: infraestrutura (repositórios HTTP), aplicação (serviços de orquestração), domínio (modelos de entidade) e apresentação (telas, widgets e providers).

Na prática, essa separação permitiu que mudanças na URL base da API, por exemplo, afetassem apenas a camada de infraestrutura, sem nenhuma alteração nas camadas de domínio ou apresentação. Também facilitou o desenvolvimento paralelo das telas e dos serviços, já que cada camada tem uma responsabilidade bem definida. A principal dificuldade observada foi o volume de arquivos gerado pela separação rigorosa em camadas, que pode parecer excessivo para projetos de pequena escala, mas que se justifica pela clareza e manutenibilidade que proporciona à medida que o sistema cresce.

6. Conclusão

O desenvolvimento do FilaFácil ao longo das quatro sprints permitiu aplicar de forma integrada os padrões e tecnologias estudados na disciplina. A combinação de REST para comunicação síncrona, RabbitMQ para eventos assíncronos e Clean Architecture para organização do código resultou em um sistema funcional, de fácil manutenção e com responsabilidades bem delimitadas entre seus componentes.

A Sprint 4 concluiu o ciclo com a entrega do aplicativo do prestador e a demonstração do fluxo completo ponta a ponta, desde a entrada do cliente na fila até a liberação da mesa após o atendimento. Os dois aplicativos Flutter rodando simultaneamente, integrados ao backend e ao RabbitMQ, validaram a arquitetura proposta desde a Sprint 1.

Referências Bibliográficas

FIELDING, Roy Thomas. Architectural Styles and the Design of Network-based Software Architectures. Tese de Doutorado — University of California, Irvine, 2000. Disponível em: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>

MARTIN, Robert C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, 2017.

HOHPE, Gregor; **WOOLF**, Bobby. Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions. Addison-Wesley, 2003.